Measurement 261 (2026) 119631



Contents lists available at ScienceDirect

Measurement

journal homepage: www.elsevier.com/locate/measurement





Homomorphism marker field: A fast, orientable, and large-scale coding scheme for absolute 3-DoF translation and rotation measurement

Zian Mao, Jiayue Ding, Guowei Shi, Peisen Huang *

Shanghai Jiao Tong University UM-SJTU Joint Institute, No. 800 Dongchuan Road, Minhang District, Shanghai, 200240, China

ARTICLE INFO

Keywords:
Fiducial markers
In-plane positioning
Self-location patterns
Pseudo-random sequences
Lempel homomorphism
Planar encoder
Phase estimation

ABSTRACT

This paper introduces the Homomorphism Marker Field (HMF), a novel coding scheme that enables fast, orientable self-localization over large 2D areas. HMF uniquely combines aperiodic orientable sequences (AOS) and Lempel's D-homomorphism, via a 2D tiling framework, allowing it to be efficiently constructed on desktop computers with exceptionally large field sizes. HMF determines unique position and orientation from a $(2n+1)$-bit cross-shaped window within fields slightly exceeding $(2^{n-1}/3) \times (2^{n-1}/3)$. It achieves a linear decoding complexity relative to the field side-length, significantly outperforming De Bruijn Torus (DBT) and Uniform Marker Field (UMF). The application of HMF to absolute 3-DoF translation and rotation measurement is also discussed, which involves a checkerboard pattern embedded with HMF and an industrial camera. A 2D Interpolated Discrete Fourier Transform (2D-IpDFT) method estimates frequency and phase for precise relative translation and rotation. This, together with HMF decoding, enables absolute pose measurement. Simulations and experiments demonstrate superior decoding speed, robustness, and precision of the proposed method, achieving Root Mean Squared Error (RMSE) of 8.5 nm in translation and 1.7 μrad in rotation. Coupled with HMF's potential for large-scale fields, absolute nanometer-level positioning over meter-scale areas becomes feasible, which opens up wide applications in precision measurement.

1. Introduction

Like mammals marking their territory with odors, marking space in a manner that local features contain information about their positions and orientations in the entire space has a long pragmatic history. In engineering applications, this problem manifests as the need to mark a plane so that the relative translation and rotation between a camera and the plane can be determined in a single frame — a concept often termed 2D fiducial marking. This term originated in the 1990s from research in robotics [1] and has since inspired the development of numerous solutions [2–4]. However, there are limitations to conventional individual tag-like fiducial markers. A conundrum often encountered is that marking a relatively large plane by simply piling or enlarging individual markers proves inadequate due to the camera's limited field of view. The space in between the individual markers also leads to reduced information density and compromised accuracy.

This is where marker fields become relevant. The general idea is to design a coding scheme on a planar, rectangular surface in such a way that, given a small part of the pattern, the exact position and orientation of this part-pattern on the surface can be determined [5]. This allows for continuous localization of a camera as it moves through the field. The aforementioned characteristics render marker fields highly suitable for diverse applications, including pose estimation [6,7], camera calibration [8], augmented reality [9], object tracking [10], robot localization [11], structured light [12], among others.

De Bruijn torus (DBT) [13], with a property that any size $n \times n$ window (sub-matrix) is unique among the whole $2^n \times 2^n$ binary DBT matrix, is a natural choice for such a coding scheme. With DBT, knowing a specific window allows us to determine its position within the entire matrix. However, this inherent uniqueness is only true to a fixed orientation (0, $\pi/2$, π, or $3\pi/2$). If a window is given with unknown orientation, multiple matches will result, rendering it impossible to uniquely determine the position and orientation of the window. A coding scheme with such a property is called non-orientable, which poses significant limitations for practical applications, particularly those involving cameras, where the orientation of the DBT pattern relative to the camera's viewpoint may not be controlled or known a priori. To solve this problem, Uniform Marker Field (UMF) was proposed by Szentandrási et al. [7], which encodes both position and orientation information, providing complete in-plane pose information from a local observation. However, generating a functional UMF, even at a size of

* Correspondence to: 800 Dongchuan Road, Minhang District, Shanghai, 200240, China.
E-mail addresses: asakura_kukii@sjtu.edu.cn (Z. Mao), dingjiayue@sjtu.edu.cn (J. Ding), shiguowei@sjtu.edu.cn (G. Shi), peisen.huang@sjtu.edu.cn (P. Huang).

https://doi.org/10.1016/j.measurement.2025.119631
Received 7 June 2025; Received in revised form 15 October 2025; Accepted 4 November 2025
Available online 1 December 2025
0263-2241/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

92×92 with a 4×4 window, demands considerable computing power, to a point that use of a supercomputer is necessary. HydraMarker based on a binary wave function collapse (bWFC) algorithm is another coding scheme developed by Zhu et al. [14] to generate orientable marker fields with drastically reduced computational time. HydraMarker performs well in piling individual markers, but only on a small scale. When the field grows much larger than the camera acquisition size, calculation of the bWFC algorithm is still exceedingly time consuming. Therefore, despite significant advances in generation efficiency, producing marker fields of very large sizes (e.g. $>1000 \times 1000$) still faces challenges due to the inherent combinatorial complexity of having to satisfy uniqueness constraints across all positions and orientations [14].

To overcome the above limitations, this paper proposes a new coding scheme named Homomorphism Marker Field (HMF) that is capable of creating fast, orientable, large-scale, and easily-generated marker fields. In the 2D binary case, HMF is a 4-orientable, self-locating matrix where a cross-shaped window (composed of n-horizontal and n-vertical bits) can be uniquely positioned within a total coverage area slightly larger than $2^{n-1}/3$ by $2^{n-1}/3$. By leveraging aperiodic orientable sequences (AOS) and Lempel's D-homomorphism within a 2D tiling framework proposed by Bruckstein et al. [15], HMF matrices of arbitrary field sizes can be efficiently generated on desktop computers and decoded with a linear complexity. Fields larger than $10{,}000 \times 10{,}000$ can be easily generated in a few minutes time.

A typical application of coding schemes is absolute translation and rotation measurement. For instance, Yu et al. [16] developed an absolute linear displacement measurement (LDM) method based on an absolute coding scheme that uses an M-sequence pseudo-random code and a subdivision algorithm. Herout et al. [6] introduced a camera pose estimation method based on an improved design of UMF with a grayscale checkerboard and an algorithm that detects the grid of squares using edge gradients. Inspired by these works, we propose a novel absolute 3-DoF (x, y, θ) translation and rotation measurement method that uses a camera and a checkerboard pattern encoded with HMF. In this method, the coarse absolute position and orientation information is obtained by decoding HMF, while the fine translation and rotation information is determined by estimating the phase of the checkerboard pattern, enabling high precision translation and rotation measurement within a large measurement range.

The unique combination of massive scalability, orientability, and fast decoding speed makes HMF a compelling solution for a range of challenging applications where traditional methods fall short. For instance, in industrial automation, HMF can be used to develop absolute encoders that measure position and rotation simultaneously, with a range of several meters and a resolution at the nanometer level using two orthogonal linear sensors. In the domain of coded structured light, HMF overcomes a critical limitation by enabling the efficient generation of vast, unique projection patterns, allowing for rapid, high-resolution 3D scanning of large-scale objects without the correspondence ambiguity that challenges conventional methods.

Section 2 introduces AOS and Lempel's D-homomorphism, along with their roles in the construction and decoding of HMF. Section 3 details the proposed method for absolute 3-DoF translation and rotation measurement. In Section 4, simulation results are shown to compare HMF against DBT and UMF in terms of decoding efficiency and robustness. Section 5 describes experimental evaluations of the performance of the absolute 3-DoF translation and rotation measurement method. Finally, Section 6 presents the conclusion.

2. Homomorphism marker field

2.1. De Bruijn sequences

An n-order de Bruijn sequence (DBS) [13] is a cyclic binary sequence in which all n-bit long sub-sequences (n-tuple windows) appear exactly once, either within a period (in the periodic case) or throughout the entire sequence (in the aperiodic case). This inherent uniqueness implies that searching for any local n-tuple window within a period or the whole sequence will yield its unique position. To incorporate all possible n-tuple windows, an n-order DBS must consist of 2^n bits. This characteristic leads to DBS being viewed as "perfect" [17] due to its optimal length.

The foundational work on DBS [13], along with related constructs such as "shift-register sequences" [18] or "pseudo-random sequences" [19], have been widely studied and practiced in engineering fields. Of particular interest is a special kind of sequence known as "orientable sequences". In these sequences, any n-tuple window exists only once, irrespective of whether it is read forwards or backwards. The significance of such "orientable sequences" was first described by Martin et al. [20] and Burns and Mitchell [5]. More recently, their properties and generation methods were summarized for both binary [21] and non-binary [22] alphabets.

2.2. Aperiodic orientable sequences

Inspired by DBS and the necessity to identify the orientation of a sequence locally, an n-order AOS is defined with the following two properties : First, any n-tuple window occurs only once in the sequence and second, for each unique n-tuple window $X = x_1 x_2 \dots x_n \subset S_n$, its reverse $X^R = x_n x_{n-1} \dots x_1$ does not exist in S_n. It can be further inferred that any symmetric n-tuple ($X = X^R$) will never appear in AOS. The basic definitions are provided below.

Definition 1. For an n-tuple window $X = x_1 x_2 \dots x_n$, its reverse is defined as $X^R = x_n x_{n-1} \dots x_2 x_1$. Its Boolean complement is defined as $\overline{X} = \overline{x_1 x_2} \dots \overline{x_n}$.

Example 1. $X = 01011, X^R = 11010, \overline{X} = 10100$.

Definition 2. A sequence S_n is said to be an n-order AOS if, for any n-tuple windows $X_i,\ X_j \subset S_n$, $X_i \neq X_j$ and $X_i \neq X_j^R$.

The maximum length of a binary AOS can be determined by the following theorem [21]:

Theorem 1. *For $r \in \mathbb{N}$, denote the maximum length of an n-order binary AOS as L_{max}. We have*

$$\begin{aligned} L_{max} &= 2^{2r-2} + (2^{2r-2} - 1)/3 + 2r - 1 \quad \gtrapprox 2^n/3, \quad \textit{when order } n = 2r \\ L_{max} &= 2^{2r-1} + (2^{2r-1} - 2)/3 + 2r \quad\quad\ \gtrapprox 2^n/3, \quad \textit{when order } n = 2r+1 \end{aligned}$$

Example 2. An AOS of order 4 can be $S_4 = 00010111$ or $S_4 = 11101000$.

Theorem 2. *Let S_n be an n-order AOS and S_n^R its reverse. It holds that no n-tuple window of S_n appears in S_n^R, and conversely, no n-tuple window of S_n^R appears in S_n.*

Hence, the uniqueness of n-tuple windows in the sequences S_n and S_n^R is assured. Consequently, it is possible to determine whether a sampled n-tuple window is a sub-sequence of S_n or S_n^R, and to ascertain its position. These facts establish the basis for the proposed marker field.

Example 3. Consider an 4-tuple window $X = 1000$ sampled from an AOS of order 4. Given that $S_4 = 00010111$ and $S_4^R = 11101000$, we can see that $X \subset S_4^R$. This indicates that the sample is taken in the reverse direction and can be found in S_4^R at the position $1110\underline{1000}$.

An AOS can be obtained using a straightforward brute force recursive search. The pseudo-code for this approach is shown in Algorithm 1. This algorithm generates the longest n-order AOS using depth-first search, with the n-tuple $X = \overbrace{00\dots01}^{n \text{ bits}}$ as the head of the sequence.

Algorithm 1 Brute Force Construction of an AOS

The input, n, represents the order of the sequence.
The output, H, represents the ordered set of n-tuple windows of the generated AOS.

INITIATE(n)
  $X = x_1x_2 \dots x_n = \overbrace{00 \dots 01}^{n \text{ bits}}$
  add X **to history** H
  H = SEARCH(X, H)
  return H

SEARCH(X, H)
  $X_0 \leftarrow [x_1, x_2, \dots, x_n, 0]$
  $X_1 \leftarrow [x_1, x_2, \dots, x_n, 1]$
  $H_0 \leftarrow H$, $H_1 \leftarrow H$
  if X_0 **and** X_0^R **is not in** H **&&** $X_0 \neq X_0^R$
    add X_0 **to history** H_0
    H_0 = SEARCH(X_0, H_0)
  if X_1 **and** X_1^R **is not in** H **&&** $X_1 \neq X_1^R$
    add X_1 **to history** H_1
    H_1 = SEARCH(X_1, H_1)
  if $size(H_0) > size(H_1)$ **return** H_0
  else return H_1

Example 4. When $n = 6$, an example of the longest sequence generated is $S_6 = 0000011101010000110100011111$.

2.3. Half de Bruijn sequence

While AOSs enable n-tuple identification and orientation, they are inherently one-dimensional. Constructing a 2D marker field that preserves 2D orientability thus requires a specialized methodology. Our proposed marker field addresses this by employing homomorphism to transform two source AOSs to two encoded sequences that can be constructed to form a 2D field, which can be decoded to recover the original AOSs. This transformation is based on Lempel's D-homomorphism and its inverse. The inverse D-homomorphism acts as a multimap, capable of generating a pair of distinct but related (n+1)-order sequences from a single n-order sequence. Conversely, the D-homomorphism allows for the recovery of the original sequence from these generated sequences. This pathway is the process of transforming a DBS into what is known as a half DBS. The purpose of introducing inverse D-homomorphism is to construct 2D marker fields from 1D AOS sequences. The significance of inverse D-homomorphism will be discussed later in Section 2.4.

Definition 3. A sequence S_n is identified as an n-order half DBS if, for any n-tuple windows X_i, $X_j \subset S_n$, it holds that $X_i \neq X_j$ and $X_i \neq \overline{X_j}$.

Example 5. A binary half DBS of order 4 is $S_4 = 000011010$

As mentioned above, a half DBS can be generated from DBS via the inverse of Lempel's D-homomorphism. The original and inverse of D-homomorphism are defined as follows:

Definition 4. $\mathbf{D}(S_n) = d_1d_2 \dots d_{N-1}$ is the result of applying D-homomorphism to $S_n = s_1s_2 \dots s_N$. For $j \in 1, 2, \dots, N$, $d_j = s_j \oplus s_{j+1}$.

Definition 5. $\mathbf{D}^{-1}(S_n) = d_1d_2 \dots d_{N+1}$ is the result of applying inverse D-homomorphism to $S_n = s_1s_2 \dots s_N$. For $j \in 1, 2, \dots, N$, $d_{j+1} = d_j \oplus s_j$.

Here, the symbol $\oplus$ represents the **XOR** operation from Boolean algebra. The resulting sequence, $\mathbf{D}^{-1}(S_n)$, is a half DBS of order $n + 1$. Lempel's inverse D-homomorphism serves to transform an n-order DBS into an $(n + 1)$-order half DBS [15].

This mapping increases the window length by 1 bit but preserves the local uniqueness. Since D-homomorphism is surjective and not bijective, inverse D-homomorphism acts as a multimap. There are two outcomes when applying $\mathbf{D}^{-1}$ to a tuple, each corresponding to d_1 equal to 0 or 1. They are notated as follows:

$$\begin{aligned} &\mathbf{D}^{-1}(S_n) = d_1d_2 \dots d_{N+1}, \text{ when } d_1 = 0 \\ &\overline{\mathbf{D}^{-1}(S_n)} = d_1d_2 \dots d_{N+1}, \text{ when } d_1 = 1 \end{aligned} \tag{1}$$

Here, we assume without loss of generality that $\mathbf{D}^{-1}(S_n)$ starts with a 0 ($d_1 = 0$). The two sequences yield the same result when D-homomorphism is applied. That is

$$\mathbf{D}(\mathbf{D}^{-1}(S_n)) = \mathbf{D}(\overline{\mathbf{D}^{-1}(S_n)}) = S_n \tag{2}$$

Example 6. Let $S_3 = 00010111$ be a DBS of order 3. For $d_1 = 0$ and 1, $\mathbf{D}^{-1}(S_3) = 000011010$ and $\overline{\mathbf{D}^{-1}(S_3)} = 111100101$, both half DBSs. Applying D-homomorphism, we obtain $\mathbf{D}(\mathbf{D}^{-1}(S_3)) = \mathbf{D}(\overline{\mathbf{D}^{-1}(S_3)}) = S_3 = 00010111$.

We now prove that this reversibility holds true not just for the entire sequence, but for any local (n+1)-tuple sub-sequences.

Lemma 1. *Let X_n be an n-tuple source window in a DBS S_n and let Y_{n+1} be its corresponding (n+1)-tuple window in either $\mathbf{D}^{-1}(S_n)$ or $\overline{\mathbf{D}^{-1}(S_n)}$. Applying the D-homomorphism to Y_{n+1} uniquely recovers the original window: $\mathbf{D}(Y_{n+1}) = X_n$.*

Proof. Let $X_n = x_1x_2 \dots x_n$ and $Y_{n+1} = y_1y_2 \dots y_{n+1}$. By Definition 5, the transformation is defined by $y_{j+1} = y_j \oplus x_j$ for $j \in \{1, \dots, n\}$. This can be rearranged to $x_j = y_j \oplus y_{j+1}$. By Definition 4, $\mathbf{D}(Y_{n+1}) = (y_1 \oplus y_2)(y_2 \oplus y_3) \dots (y_n \oplus y_{n+1}) = x_1x_2 \dots x_n = X_n$. □

This property extends directly to AOSs. An AOS (see Definition 2) ensures that each n-tuple sub-sequence X_n is unique and, critically, distinct from its reverse, X_n^R. We now prove that our mapping preserves this orientable property.

Lemma 2. *Let X_n be an n-tuple source window in an AOS S_n and let Y_{n+1} be its corresponding (n+1)-tuple window in either $\mathbf{D}^{-1}(S_n)$ or $\overline{\mathbf{D}^{-1}(S_n)}$. Applying the D-homomorphism to Y_{n+1} and its reverse Y_{n+1}^R uniquely recovers the original source window and its reverse: $\mathbf{D}(Y_{n+1}) = X_n$ and $\mathbf{D}(Y_{n+1}^R) = X_n^R$.*

Proof. Let $X_n = x_1x_2 \dots x_n$ and $Y_{n+1} = y_1y_2 \dots y_{n+1}$. By Definition 5, the transformation is defined by $y_{j+1} = y_j \oplus x_j$ for $j \in \{1, \dots, n\}$. This can be rearranged to $x_j = y_j \oplus y_{j+1}$. By Definition 4, $\mathbf{D}(Y_{n+1}) = (y_1 \oplus y_2)(y_2 \oplus y_3) \dots (y_n \oplus y_{n+1}) = x_1x_2 \dots x_n = X_n$. $\mathbf{D}(Y_{n+1}^R) = (y_n \oplus y_{n+1})(y_{n-1} \oplus y_n) \dots (y_1 \oplus y_2) = x_nx_{n-1} \dots x_1 = X_n^R$. □

Lemma 2 guarantees that given an AOS S_n, an (n+1)-tuple window from either $\mathbf{D}^{-1}(S_n)$ or $\overline{\mathbf{D}^{-1}(S_n)}$ can be uniquely decoded to its source n-tuple window and its orientation (forward or reverse) within the original AOS, thereby unambiguously determining both its position and orientation.

The two distinct transformed sequences, $\mathbf{D}^{-1}(S_n)$ and its complement $\overline{\mathbf{D}^{-1}(S_n)}$, thus serve as information carriers for a second, orthogonal dimension. By arranging these two sequences along this new axis in an encoded manner, separable decoding can be achieved. Crucially, this is accomplished while preserving the shift-register property along both orthogonal axes. This principle of orthogonal, separable decoding is fundamental to the 2D construction that follows.

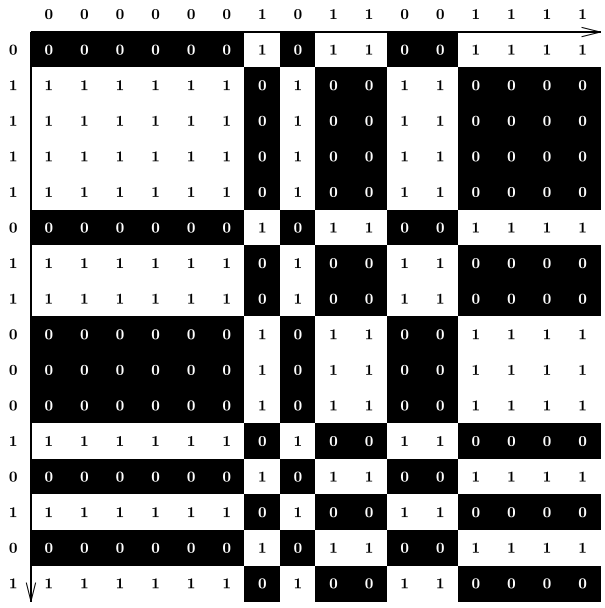


Fig. 1. The constructed 2-D marker field. $G_{6+1} = \mathbf{D}^{-1}(X_6) \otimes \mathbf{D}^{-1}(Y_6)$.

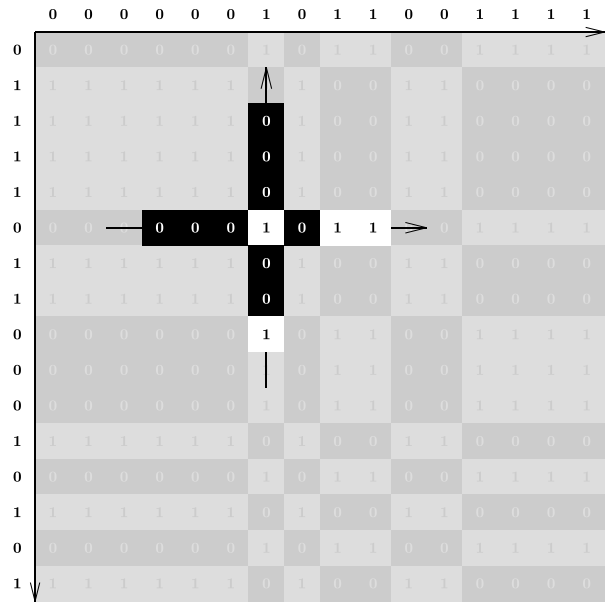


Fig. 2. Two perpendicular segments are selected from the 2-D marker field G_{6+1}. The sampling directions are randomly selected and given.

2.4. Construction

To construct the proposed 2-D HMF, an AOS, S_n of order n, is generated, from which two sub-sequences X_n and Y_n are selected. These two sub-sequences can overlap by at most $n-1$ bits. We then apply inverse D-homomorphism to X_n and Y_n, yielding $\mathbf{D}^{-1}(X_n)$ and $\mathbf{D}^{-1}(Y_n)$, respectively.

Example 7. Plugging $n = 6$ into Algorithm 1, we obtain $S_6 = 000001110101000011010011111$. Let $X_6 = 000001110101000$ and $Y_6 = 100011010011111$. Applying inverse D-homomorphism to X_6 and Y_6, we have

$$\begin{aligned}\mathbf{D}^{-1}(X_6) &= 0000010110011111\\ \mathbf{D}^{-1}(Y_6) &= 0111101100010101\end{aligned} \tag{3}$$

The proposed marker field is then constructed using the product construction method with the operator $\otimes$ [15], which is defined as follows:

Definition 6. For two sequences $A = a_1 a_2 \ldots a_M$ and $B = b_1 b_2 \ldots b_N$, $A \otimes B$ is a $M \times N$ array G with $g_{i,j} = a_i \oplus b_j$, $1 \le i \le M$, $1 \le j \le N$.

In the generated array G_{n+1}, each row corresponds to $\mathbf{D}^{-1}(X_n)$ or $\overline{\mathbf{D}^{-1}(X_n)}$ and each column corresponds to $\mathbf{D}^{-1}(Y_n)$ or $\overline{\mathbf{D}^{-1}(Y_n)}$, as illustrated in Fig. 1. In practical applications, we typically split one AOS into two sub-sequences, each for the coding along the X- or Y-axis respectively. Naturally the maximum area of coverage is obtained when lengths of X and Y are equal, resulting in a maximum area of G_{n+1} approximately equal to $(2^{n-1}/3 \times 2^{n-1}/3)$.

Following our discussion in Section 2.3, the importance of inverse D-homomorphism lies in the fact that in order to construct a 2D array with boolean XOR $\otimes$, the prior process of applying inverse D-homomorphism is necessary, which allows for correct information extraction in both the original sequence and its logical complement. In the constructed G_{n+1}, two orthogonal (n+1)-tuples are sampled that belongs to $\mathbf{D}^{-1}(X_n)$ or $\overline{\mathbf{D}^{-1}(X_n)}$, $\mathbf{D}^{-1}(Y_n)$ or $\overline{\mathbf{D}^{-1}(Y_n)}$ respectively, which can be further decoded to retrieve position and orientation information.

2.5. Decoding

From G_{n+1}, we select a cross-shaped window (see Fig. 2) comprising two perpendicular segments of length $n+1$. Its uniqueness across the array is guaranteed as demonstrated in [15]. Below we detail the decoding process using this cross-shaped window.

Denote the two sampled segments as $\mathbf{D}^{-1}(A_n)$ and $\mathbf{D}^{-1}(B_n)$ respectively. We can apply D-homomorphism to them to obtain A_n and B_n, as shown in the following example:

Example 8. From Fig. 2, we obtain $\mathbf{D}^{-1}(A_6) = 1001000$ and $\mathbf{D}^{-1}(B_6) = 0001011$. Applying D-homomorphism yields

$$\begin{aligned}\mathbf{D}(\mathbf{D}^{-1}(A_6)) &= \mathbf{D}(1001000) = 101100 = A_6\\ \mathbf{D}(\mathbf{D}^{-1}(B_6)) &= \mathbf{D}(0001011) = 001110 = B_6\end{aligned} \tag{4}$$

A_n and B_n are both n-tuple windows of X_n, X_n^R, Y_n, or Y_n^R, sampled along one direction of different axes. Conducting 1-D searches for A_n and B_n in X_n, X_n^R, Y_n, and Y_n^R determines each window's corresponding axis, direction, and position. With this information at hand, we can interpret the sampling directions of A_n and B_n as two base vectors defining the sampling coordinate system. This understanding then provides a clear linkage to the coordinate system of the marker field, an essential next step in our examination.

The following example demonstrates how analyzing sampled segments A_6 and B_6 allows for position and orientation recovery:

Example 9. Searching A_6 and B_6 in the reference sequences (X_6, X_6^R, Y_6, and Y_6^R) identifies their locations: $A_6 \subset Y_6^R$ and $B_6 \subset X_6$. The positions of these segments in Y_6^R and X_6 serve as indices that map directly to the position within the marker field where the original data was sampled. Using these mapped positions, adjusted for the segment length, the crossing point is determined to be $(+6, +5)$ blocks from the marker field origin $(1, 1)$. Furthermore, the specific combination of sequences where the segments were found (Y_6^R for A_6 and X_6 for B_6) uniquely indicates that the sampling coordinate system is rotated $\frac{\pi}{2}$ counter-clockwise relative to the original HMF.

$$\begin{aligned}A_6 &= 101100\\ B_6 &= 001110\\ X_6 &= 000\underline{001110}101000\\ X_6^R &= 000101011100000\\ Y_6 &= 100011010011111\\ Y_6^R &= 1111100\underline{10110}001\end{aligned} \tag{5}$$

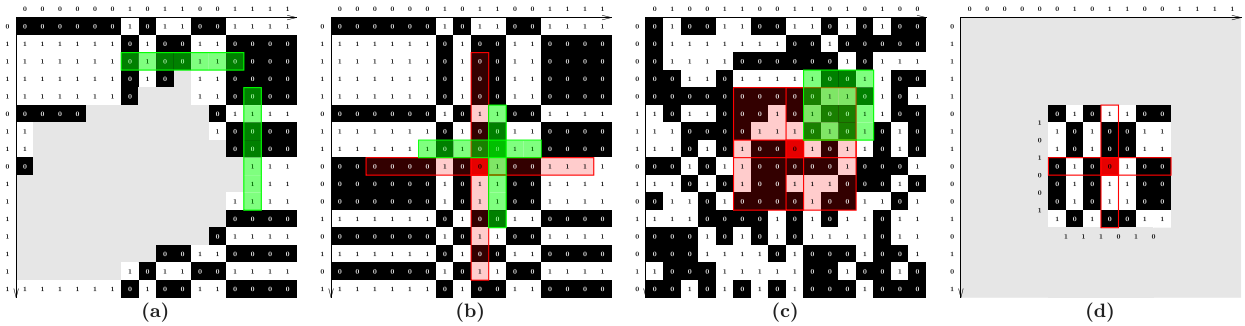


Fig. 3. HMF robustness mechanisms. (a) Flexible decoding allows for pose recovery from any visible orthogonal row and column (green) even with significant occlusion (gray). (b) A single bit error (red) in HMF creates a cross-shaped non-decodable area, with decodable window (green) near the error bit. (c) In contrast, the same error in a square-based marker creates a much larger, contiguous non-decodable area. (d) The majority voting mechanism uses redundant information in a captured window to identify and discard corrupted rows or columns (red), ensuring a reliable decoding result.

Hence, the position and orientation of a sample can be determined from the cross-shaped window. We can summarize the steps for constructing and decoding HMF as follows:

Summary of HMF construction procedure

One n-order AOS of length $l \geq 2k$ is generated and divided into two sub-sequences of length $k+n-1$ with an overlap of $n-1$ bits. Applying inverse D-homomorphism [23] gives two sequences of equal length $k+n$. With the two new sequences as the X and Y axes, a 2-D array G can be generated as $G = X \otimes Y$.

Summary of HMF decoding procedure

Take a "cross-shaped window", including $n+1$ consecutive horizontal and vertical bits with known relative displacement, and apply D-homomorphism respectively. The resulting two n-bit sequences are unique in the original AOS. The window's position and orientation can be obtained through 1-D searching in the AOS.

2.6. Robustness mechanism

In practical applications, the image captured by a camera typically samples an area of the marker field significantly larger than the minimum window required for a single decoding operation. This inherent oversampling creates redundant information that can be exploited to achieve robust decoding in the presence of real-world imperfections such as partial occlusion, imaging artifacts, or physical defects on the marker surface. The geometry of the HMF window represents a deliberate design trade-off, while cross-shaped window might seem more susceptible to occlusion than a compact square, it enables a uniquely flexible and powerful set of error-handling mechanisms that are not available to square-based markers.

For handling partial occlusion, HMF utilizes a flexible decoding strategy. Decoding does not require a fixed, contiguous cross-shape but can be performed using any visible and complete row and column pair within the image, provided their relative displacement is known. An explanatory figure is provided in Fig. 3(a). This approach allows the system to bypass localized occlusions and reduces the non-decodable "shadow" caused by errors, which would otherwise invalidate a larger area in compact square-based markers (see Fig. 3(b) (c)). Quantitative simulation results are provided in Section 4.1.2, validating that the HMF's small non-decodable area provides higher robustness in a fair comparison.

To combat sporadic bit-flips from noise or minor defects, a computationally efficient majority voting mechanism is employed. D-homomorphism is applied to all available rows and columns in the captured image. Since all valid rows (or columns) must decode to the same source n-tuple (see Fig. 3), the most frequently occurring n-tuple is selected as the correct one, effectively filtering out anomalous results. Such voting mechanism does not require re-searching in the AOS sequences, providing higher robustness with zero performance loss.

An additional error-checking layer is provided by the mathematical properties of AOS. By definition, an AOS contains no symmetric n-tuples. Therefore, if a decoded sequence is found to be symmetric, it is immediately identified as corrupted and is discarded from the voting process, further enhancing the reliability of the decoding.

3. Absolute 3-DoF translation and rotation measurement

Phase estimation methods are commonly used to determine the phase information of a periodic pattern for accurate measurement of translations and rotations [24–29]. However, these methods provide phase information only on a sub-period scale and thus cannot be used alone to determine absolute pose information. On the other hand, coding schemes can provide absolute pose information but only at a resolution of the period. Therefore, by embedding the codes in the periodic pattern and combining the information given by both phase estimation methods and coding schemes, accurate absolute pose information can be obtained.

As an application of the proposed HMF, we explore an absolute 3-DoF pose measurement method, which is characterized by the following steps. First, HMF codes are embedded in a checkerboard, forming a composite pattern. Second, a 2D interpolation discrete Fourier transform (2D-IpDFT) method [30], extended from a 1D-IpDFT method [31], is applied to estimate the frequency and phase information of the acquired pattern. Third, the HMF codes are decoded to find the position and orientation of the acquired window array in the marker field. Finally, the results obtained in Steps 2 and 3 are combined to determine the accurate absolute 3-DoF pose of the camera relative to the marker field.

3.1. Coding

Fig. 4 shows a checkerboard pattern with an embedded HMF of a window array size $6+6$. To embed the HMF codes in a checkerboard pattern, small squares with inverse colors are added at the centers of selected checkerboard blocks, representing the code bit of 1. Code blocks that do not have small squares represent the code bit of 0. A code pitch (the number of blocks between the coded blocks) of 4 is chosen, which means that one block in every 4×4 blocks carries code information. Each embedded small square has a size of 2/5 of the size of the checkerboard blocks, which is small enough to cause only negligible impact on frequency and phase estimation. As a comparison, Fig. 5 shows a checkerboard pattern with an embedded DBT of a window array size 3×3. Since DBT is not orientable, extra orientation codes (2 neighboring coded blocks) are added. It can be seen that for the same amount of coded information, HMF needs less coded blocks than DBT and therefore would cause less impact on frequency and phase estimation.

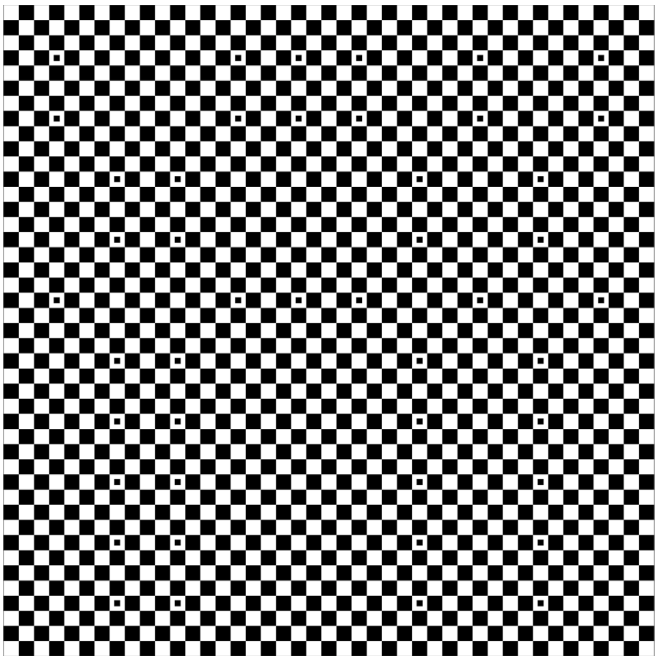


Fig. 4. A checkerboard pattern with embedded HMF codes of window array size 6 + 6. The code blocks with small squares at the center represent the code bit of 1 and the code blocks that do not have the small squares represent the code bit of 0. The code pitch is 4.

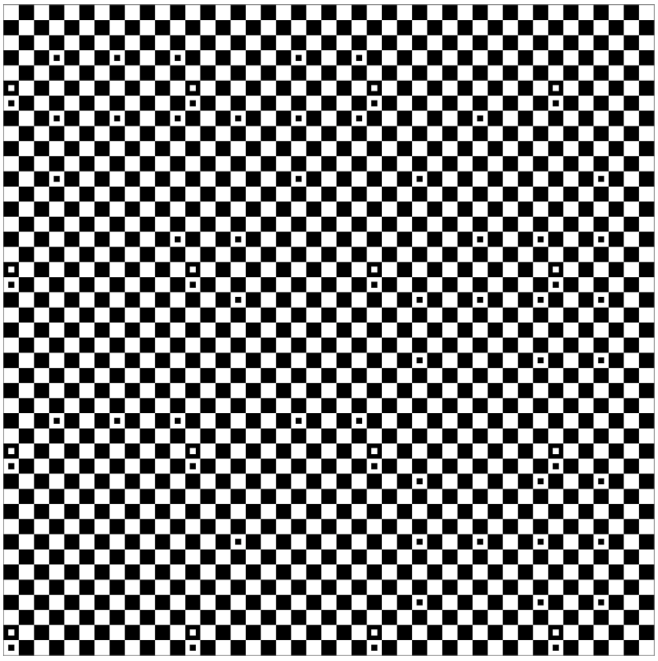


Fig. 5. A checkerboard pattern with embedded DBT codes of window array size 3 × 3. The code pitch is also 4.

3.2. Relative translation and rotation

Fig. 6 is an example image of a checkerboard pattern embedded with HMF codes. Applying the 2D Fast Fourier Transform (FFT) gives the frequency spectrum of the image, as shown in Fig. 7. The two peak points (A and B) represent two perpendicular planar sinusoids at the checkerboard's base frequencies. Their coordinates, $A(-l_1,\ l_2)$ and $B(l_2,\ l_1)$, and phase values, φ_A and φ_B, can be accurately estimated using the aforementioned 2D-IpDFT method. It should be noted that φ_A and φ_B represent the phase values of the center point of the checkerboard image. From these coordinates and phase values, the relative translation and rotation of the checkerboard can be determined.

For computational consistency, we always denote the peak point in the second quadrant as A and the peak point in the first quadrant as B. Assume that the acquired image is square-shaped with a size of $N \times N$ and the rotation is confined only to the plane. As illustrated in Fig. 7, we define a coordinate system $Ox'y'$ in the spatial domain, with its x' and y' axes aligned with the edges of the checkerboard pattern and its origin O located at the image center. We also define $\overrightarrow{V_A}$ and $\overrightarrow{V_B}$ as the diagonal vectors and $\overrightarrow{V_{x'}}$ and $\overrightarrow{V_{y'}}$ as the side vectors of the checkerboard squares. Given the coordinates of the peak points A and B in the frequency domain, we can derive these vectors as follows:

$$\begin{aligned}
\overrightarrow{V_A} &= \left[\frac{-Nl_1}{l_1^2+l_2^2},\ \frac{Nl_2}{l_1^2+l_2^2}\right] \\
\overrightarrow{V_B} &= \left[\frac{Nl_2}{l_1^2+l_2^2},\ \frac{Nl_1}{l_1^2+l_2^2}\right] \\
\overrightarrow{V_{x'}} &= \left[\frac{N(l_2-l_1)}{2l_1^2+2l_2^2},\ \frac{N(l_1+l_2)}{2l_1^2+2l_2^2}\right] \\
\overrightarrow{V_{y'}} &= \left[\frac{-N(l_1+l_2)}{2l_1^2+2l_2^2},\ \frac{N(l_2-l_1)}{2l_1^2+2l_2^2}\right]
\end{aligned} \tag{6}$$

The relative translation is defined as $(x'_{\mathrm{OC}}, y'_{\mathrm{OC}})$, where C is the center of the white block nearest to O (see Fig. 8). Since all centers of the white blocks on the checkerboard have the phase of (0, 0) for the base frequencies, we can use the phase of the image center (φ_A, φ_B) to determine the translation. That is

$$\begin{bmatrix} x'_{\mathrm{OC}} \\ y'_{\mathrm{OC}} \end{bmatrix} = \begin{bmatrix} -\dfrac{\varphi_A+\varphi_B}{2\pi}\cdot 2|\overrightarrow{V_{x'}}| \\ -\dfrac{\varphi_A-\varphi_B}{2\pi}\cdot 2|\overrightarrow{V_{y'}}| \end{bmatrix} \tag{7}$$

The rotation is defined as the angle θ' between $Ox'y'$ and the original image coordinate system and can be determined as (see Fig. 7)

$$\theta' = \arg\left(\overrightarrow{V_{x'}}\right) \tag{8}$$

3.3. Decoding

Once the relative translation and rotation of the checkerboard are determined, we can now decode the HMF codes to obtain the absolute position and orientation of the checkerboard. The decoding of the HMF codes begins by locating the center of a white block nearest to the center of the image using the phase information. Starting from this white block, we add multiples of $\overrightarrow{V_{x'}}$ and $\overrightarrow{V_{y'}}$ to find the nearest block with an inverted center (block coded as 1). We denote this block bias as $(x'_{\mathrm{CD}}, y'_{\mathrm{CD}})$ in the coordinate system of $Ox'y'$ in units of pixels. Once this block is located, we can find every single coded block in the acquired image regardless of whether it is a 1 or 0 block. In the case of HMF, cross-shaped window arrays are extracted and decoded to determine their absolute position and orientation. Similarly, for UMF and DBT, their respective rectangular window arrays are extracted by obtaining rectangular arrays.

It should be noted that the design of HMF guarantees that for any HMF of window array size $n + n$, at least one block representing 1 can always be found within any $n \times 1$ array. To prove this to be true, assume that in an HMF of window array size $n + n$, there are $n \times 1$ blocks representing 0. The D-homomorphism gives the following sequence:

$$\mathbf{D}(\overbrace{00\ldots00}^{n}) = \overbrace{00\ldots00}^{n-1} \in X_{n-1} \text{ or } Y_{n-1} \tag{9}$$

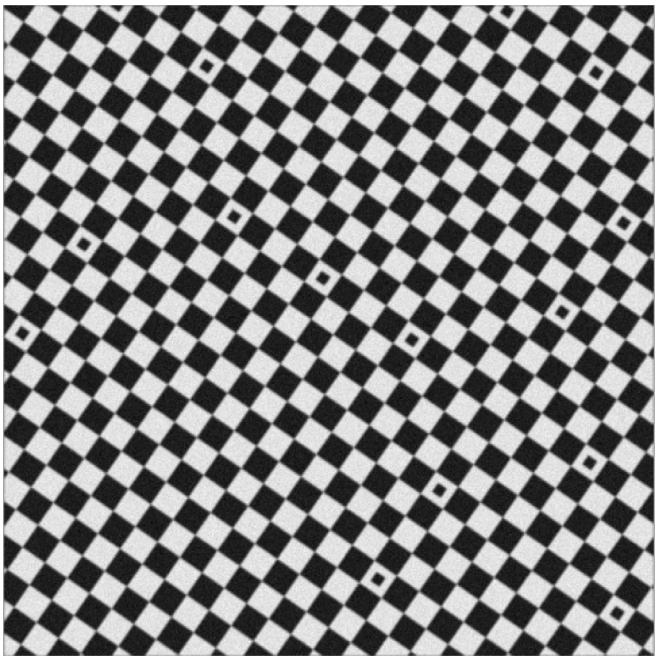


Fig. 6. Simulated image of a checkerboard pattern with embedded HMF codes. The signal-to-noise ratio (SNR) is 15 and the radius of the circular averaging filter is 2 pixels.

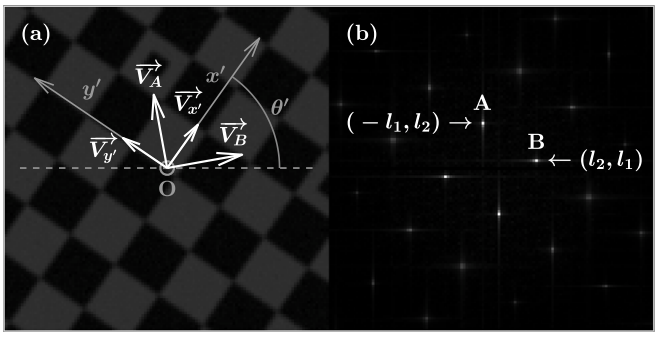


Fig. 7. The simulated image and its spectrum of the checkerboard pattern with embedded HMF codes. (a) The image of the checkerboard pattern. (b) The spectrum of the checkerboard pattern. The four brightest points represent two perpendicular planar sinusoids of the checkerboard's base frequencies. Two of these points (A and B) are selected for frequency and phase estimation.

Because X_{n-1} and Y_{n-1} are aperiodic orientable sequences, no symmetric (n-1)-tuple window should occur in them, and yet $\overbrace{00\ldots00}^{n-1}$ is symmetric, leading to a contradiction. Hence, a block representing 1 can always be found within any $n \times 1$ array.

The procedure for extracting the HMF codes embedded in the checkerboard pattern is shown in Fig. 8. It starts by finding the center of the acquired image, the center of the nearest white block, and the center of the nearest coded block. Then the cross-shaped HMF codes centered at the nearest coded block are extracted ($a_{n+1}^{-1} = 1101100$ and $b_{n+1}^{-1} = 1101001$ for this example image). Finally, these codes are used to determine the absolute position and orientation of the checkerboard by identifying their unique locations within the entire marker field.

3.4. Absolute translation and rotation

Denote the absolute position and orientation of the decoded marker field as $(x_{\text{hmf}},\ y_{\text{hmf}},\ \theta_{\text{hmf}})$. To express the position x_{hmf} and y_{hmf} in pixel units, we utilize the vectors $\overrightarrow{V_{x'}}$ and $\overrightarrow{V_{y'}}$, allowing us to calculate the number of pixels per block. The orientation θ_{hmf} is in the set of $\{0, \pi/2, \pi, 3\pi/2\}$. The absolute pose or position and rotation $(x_{\text{abs}},\ y_{\text{abs}},\ \theta_{\text{abs}})$ can be obtained as follows:

$$\begin{bmatrix} x_{\text{abs}} \\ y_{\text{abs}} \end{bmatrix} = \begin{bmatrix} x_{\text{hmf}} \\ y_{\text{hmf}} \end{bmatrix} + \begin{bmatrix} \cos\theta_{\text{hmf}} & \sin\theta_{\text{hmf}} \\ -\sin\theta_{\text{hmf}} & \cos\theta_{\text{hmf}} \end{bmatrix} \begin{bmatrix} -x'_{\text{OC}} - x'_{\text{CD}} \\ -y'_{\text{OC}} - y'_{\text{CD}} \end{bmatrix} \tag{10}$$
$$\theta_{\text{abs}} = \theta_{\text{hmf}} + \theta'$$

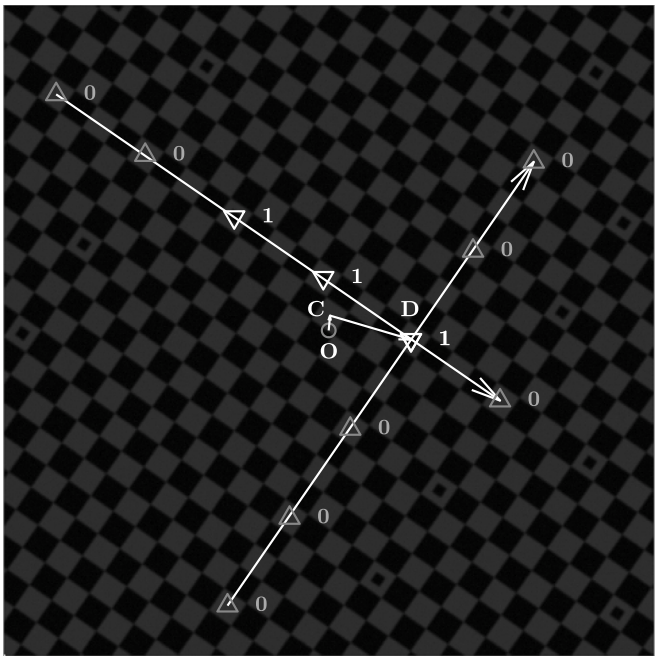


Fig. 8. Procedure for the extraction of the HMF codes embedded in the checkerboard pattern.

4. Simulations

Simulations are conducted to evaluate the performance of HMF in comparison with those of UMF and DBT. In the first simulation, HMF codes are directly decoded to study both its decoding complexity and robustness. In the second simulation, HMF, UMF, and DBT codes embedded in the checkerboard pattern are decoded. The image acquisition of these composite patterns are simulated with added Gaussian noise, defocus, as well as random translations and rotations. Subsequently, the 2D-IpDFT method is employed to estimate the frequency and phase information of the generated images, from which the relative translations and rotations are derived. Finally, the codes of the marker fields are decoded to determine the absolute 3-DoF pose of the camera. These simulations provide comparisons of the performances between HMF, UMF, and DBT. All simulations are conducted using MATLAB and executed on an AMD Ryzen 5900X platform.

4.1. Decoding of marker fields

4.1.1. Decoding time

In this simulation, HMF, UMF, and DBT are stored in computer memory and randomly sampled. Decoding simulations, executed by a MATLAB script, are designed to determine absolute positions and orientations without the need for pre-calculated hash maps, offering a direct measure of decoding complexities. Since DBT is not 4-orientable, the search is limited to positions, rather than including both positions and orientations. The simulation results can be found in Fig. 9 and Table 1.

Table 1
Decoding times of HMF, UMF, and DBT for different marker field sizes.

Marker field size[a]	UMF[b]		DBT		HMF	
(bit × bit)	Window size (bit × bit)	Decoding time (ms)	Window size (bit × bit)	Decoding time (ms)	Window size (bit + bit)	Decoding time (ms)
16 × 16	4 × 4	0.066	3 × 3	0.064	7 + 7	0.007
32 × 32	4 × 4	0.316	4 × 4	0.273	9 + 9	0.014
64 × 64	4 × 4	1.400	4 × 4	1.217	10 + 10	0.034
128 × 128	/	/	4 × 4	5.229	11 + 11	0.074
256 × 256	/	/	4 × 4	22.005	12 + 12	0.144
512 × 512	/	/	5 × 5	94.260	13 + 13	0.298
1024 × 1024	/	/	5 × 5	362.861	14 + 14	0.597
2048 × 2048	/	/	5 × 5	1468.165	15 + 15	1.252
4096 × 4096	/	/	5 × 5	6997.193	16 + 16	2.494

[a] For each field size and marker field, a minimum required window array size is selected. The decoding time is the average decoding time obtained from 10,000 simulations.
[b] The UMF presented by Szentandrási et al. [7] has a maximum size of 110 × 78.

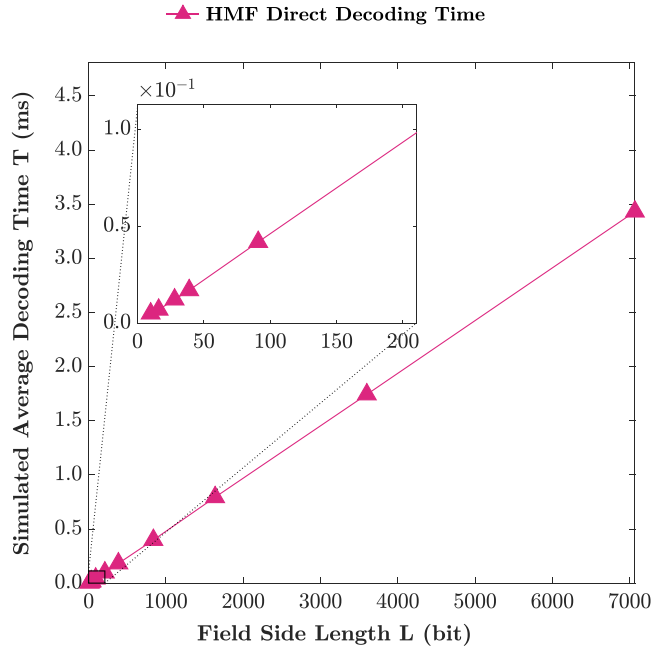


Fig. 9. Linear relationship between HMF's decoding time T and side length L of the marker field.

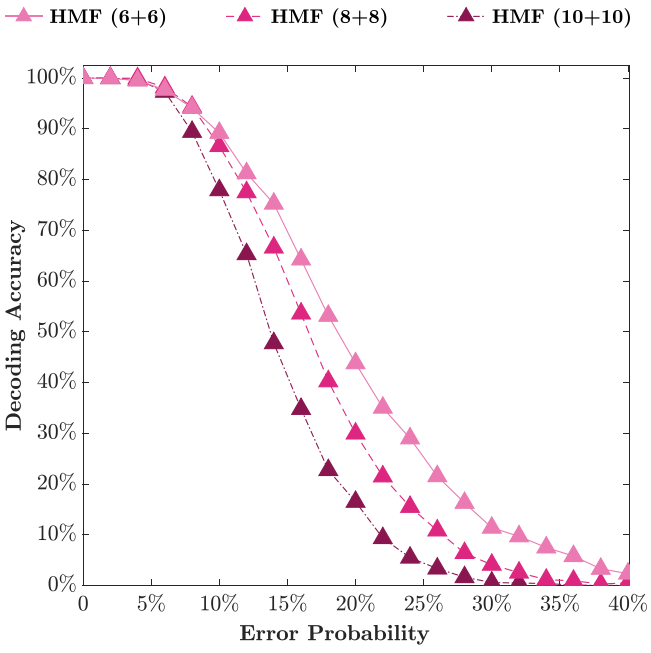


Fig. 10. Relationship between HMF's decoding accuracy and error probability in all coded blocks.

Fig. 9 shows the relationship between HMF's decoding time T and side length L of the marker field (field size $L \times L$). For each field size, a minimum required window array size is selected. Obviously, the decoding time has a linear relationship with the field side length, which indicates that the decoding of HMF is highly efficient. For example, a large HMF with a window array size 16+16 and a field size 7069×7069 can be decoded in a mere 3.432 ms.

Table 1 compares decoding times of HMF, UMF, and DBT for different field sizes. For each field size and marker field, a minimum required window array size is selected. It should be noted that DBT requires an additional decoding step for the extra orientation codes. According to Table 1, DBT's decoding time increases exponentially with field size. UMF shows a similar exponential growth in decoding time as the field size increases, although its available field sizes are limited. In contrast, HMF's decoding time increases linearly with field size and remains in the range of a few milliseconds even for a large field size of 4096×4096. These simulation results clearly demonstrate the advantage of HMF in terms of reducing decoding time and enabling fast decoding of large marker fields.

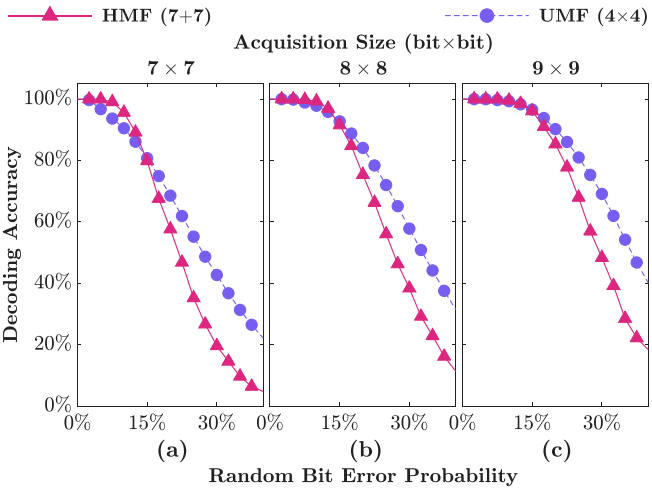


Fig. 11. Comparison of decoding accuracy between HMF and UMF under increasing levels of random bit errors for different acquisition sizes.

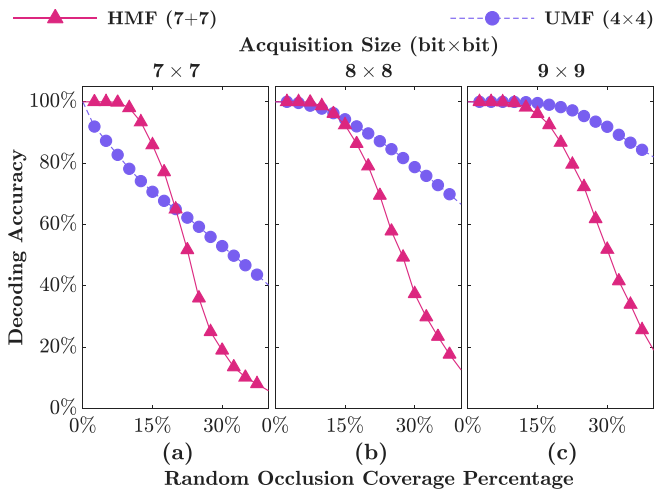


Fig. 12. Comparison of decoding accuracy between HMF and UMF under increasing levels of partial occlusion for different acquisition sizes.

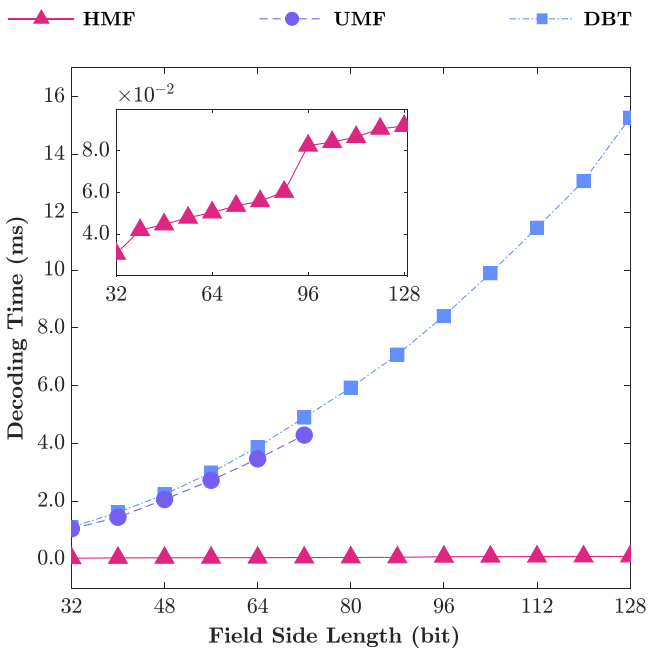


Fig. 13. Comparisons of decoding time between embedded HMF, UMF, and DBT.

4.1.2. Robustness

Following the discussion in Section 2.6, we show simulation results on HMF's performance given random bit-errors or partial occlusions.

For random bit-errors, a simulation was conducted to quantitatively evaluate the decoding performance of HMF. We model the error as an inverted bit and apply a consistent error probability to each bit within randomly sampled, minimal rectangular arrays. For several window array sizes (6+6, 8+8, and 10+10), the decoding accuracy was calculated by averaging the results of 1000 trials for each error probability level, utilizing the full majority voting and AOS self-checking mechanisms.

As can be seen from Fig. 10, HMF demonstrates a strong ability to recover from errors, maintaining a decoding accuracy of over 90% even when 10% of the bits are inverted, an error level rarely encountered in practice. In theory, for an HMF of window array size $n + n$ and an image size containing $n \times n$ codes, the decoding result is guaranteed to be correct as long as less than half of the rows and columns contain errors. This guarantee exists because, under this condition (less than 50% errors per dimension), the original, correct sequence for any given row or column will still appear more frequently than any single erroneous sequence. The HMF identifies this most frequent sequence as the correct one. In practical scenarios where error bits can be detected using other methods, the decoding accuracy can be further improved.

We further established a fair comparison between HMF and UMF to evaluate decoding robustness. For a fixed marker field size, we used the minimum required window size (e.g., HMF $(7 + 7)$ vs. UMF (4×4)) to encode position and orientation. Acquisition sizes of 7×7, 8×8, and 9×9 were selected and simulated to study HMF and UMF's performance under different redundancy levels. Random bit errors were simulated with a controlled probability. Partial occlusion was simulated by generating a random convex polygon, where the total coverage of occluded bits was controlled. For HMF, all robustness mechanisms were enabled. For UMF, an exhaustive search was performed for all 4×4 windows and a majority vote on the results determined the final pose.

The results are depicted in Figs. 11 and 12. In the case of lower redundancy levels (7×7 acquisition size), HMF shows better robustness for small bit error probability and small occlusion coverage. For square-shaped windows like UMF, a small amount of error can generate a large non-decodable area (see Fig. 3), whereas the cross-shaped window gains an advantage due to its flexible decoding. However, at higher redundancy levels, UMF's performance becomes more robust due to its synergistic properties of sparse coding and compactness. The sparse coding ensures that a window corrupted by errors has a very low probability of matching another valid pattern in the pre-defined set. This allows invalid windows to be confidently rejected as false positives, which provides advantage for a majority voting mechanism.

It should be mentioned that although HMF shows higher robustness at lower redundancy levels, it still requires a larger minimal rectangular window to guarantee successful decoding. This represents a fundamental design trade-off where HMF prioritizes large-scale coding capability and fast, linear-time decoding over maximal information density per unit area. Further work could focus on optimizing the cross-shaped window into more compact geometries, aiming to improve local information density while preserving the efficient properties of the homomorphism-based construction.

4.2. Decoding of checkerboard patterns

In this subsection, the efficiency of decoding checkerboard the patterns embedded with HMF, UMF, or DBT codes is examined through simulations. The impact of the embedded codes on the accuracy of the absolute 3-DoF pose measurement method discussed in Section 3 is also studied.

4.2.1. Decoding time

In this simulation, checkerboard patterns embedded with HMF, UMF, and DBT codes are created with a code pitch of 5 blocks and a block size of 10×10 pixels. The captured images have a fixed size of 640×640 pixels, containing approximately 64×64 blocks with 12×12 coded blocks embedded within them. For each field size and marker field, a minimum required window array size is selected. The simulation results are presented in Table 2 and Fig. 13. As shown, HMF demonstrates a significant advantage in decoding time over both UMF and DBT.

4.2.2. Impact on accuracy

In this simulation, a fixed field size of 72×72 bits is assumed. For each marker field, a minimum required window array size is selected. The image size is changed from 640×640 to 1440×1440 pixels in 10 steps. For each image size, the marker field is sampled 10,000 times with random translations $(x,\ y)$ and rotations (θ). Frequency and phase estimation is conducted based on the methodology described in

Table 2
Decoding times of HMF, UMF, and DBT embedded in checkerboard patterns for different marker field sizes.

Marker field size[a]	UMF[b]		DBT		HMF	
(bit × bit)	Window size (bit × bit)	Decoding time (ms)	Window size (bit × bit)	Decoding time (ms)	Window size (bit + bit)	Decoding time (ms)
32 × 32	4 × 4	1.048	4 × 4	1.104	9 + 9	0.031
40 × 40	4 × 4	1.447	4 × 4	1.617	10 + 10	0.042
48 × 48	4 × 4	2.061	4 × 4	2.238	10 + 10	0.045
56 × 56	4 × 4	2.726	4 × 4	2.994	10 + 10	0.048
64 × 64	4 × 4	3.468	4 × 4	3.875	10 + 10	0.050
72 × 72	4 × 4	4.291	4 × 4	4.900	10 + 10	0.054
80 × 80	/	/	4 × 4	5.932	10 + 10	0.056
88 × 88	/	/	4 × 4	7.074	10 + 10	0.060
96 × 96	/	/	4 × 4	8.410	11 + 11	0.083
104 × 104	/	/	4 × 4	9.893	11 + 11	0.084
112 × 112	/	/	4 × 4	11.464	11 + 11	0.087
120 × 120	/	/	4 × 4	13.083	11 + 11	0.091
128 × 128	/	/	4 × 4	15.273	11 + 11	0.092

[a] For each field size and each marker field, a minimum required window array size is selected. The decoding time represents the average value derived from 10,000 simulations. All marker fields are embedded into checkerboards with a code pitch of 5 blocks and a block size of 10 × 10 pixels. With an acquisition size of 640 × 640 pixels, roughly 64 × 64 blocks and 12 × 12 coded blocks are captured.
[b] The UMF presented by Szentandrási et al. [7] has a maximum size of 110 × 78.

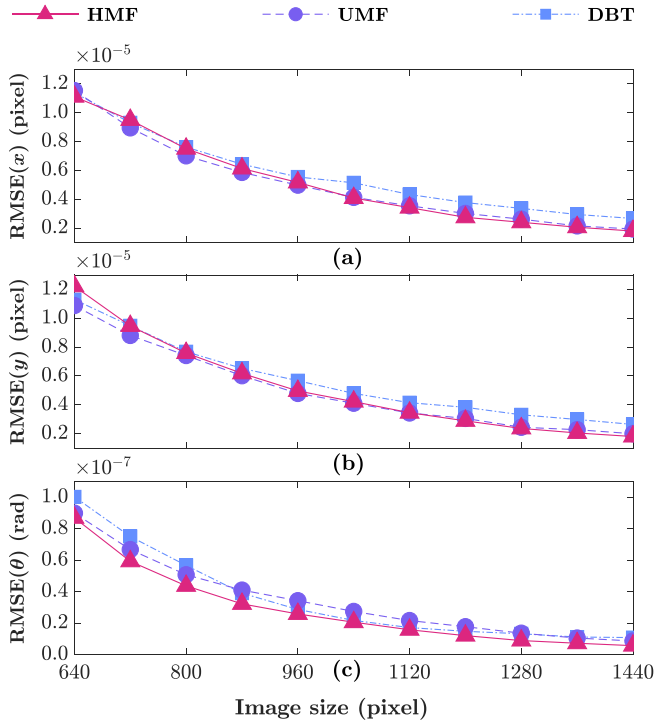


Fig. 14. Translation and rotation errors caused by different coding schemes.

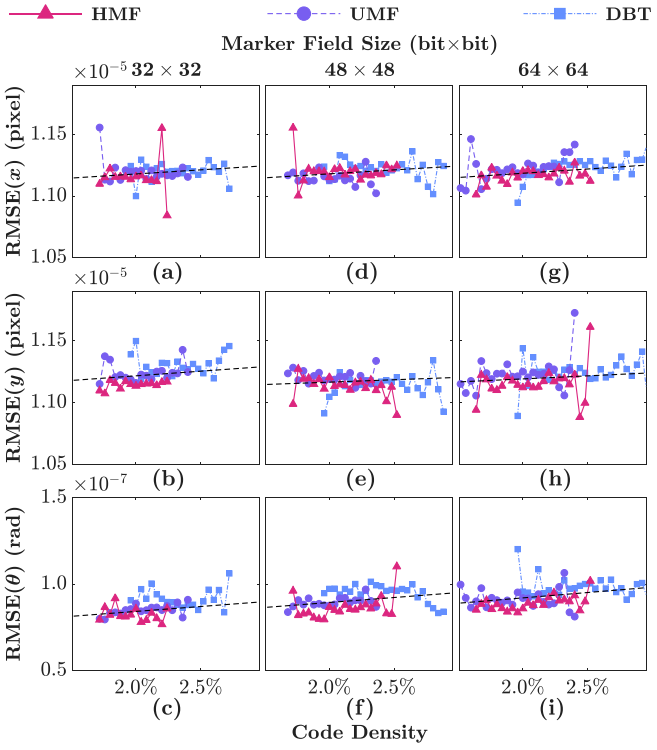


Fig. 15. Comparison of translation and rotations error as functions of the local code density. The local code density is evaluated by the number of embedded blocks divided by total visible checkerboard blocks in an acquisition window.

Section 3 to determine x, y, and θ. Root mean square errors (RMSEs) of x, y, and θ are calculated to evaluate the impact of HMF, UMF, and DBT codes on the accuracy of translation and rotation measurement. A checkerboard pattern without embedded codes serves as the control baseline.

From the results shown in Fig. 14, we see that overall, the errors decrease exponentially as the image size increases for all coding schemes. Compared to UMF and DBT, HMF exhibits the smallest error for rotation measurement regardless of the image size. However, for translations, the errors are smaller than those of UMF and DBT only when the image size is large. Embedding codes into the checkerboard patterns is equivalent to adding extra square planar waves to the captured image, with a base frequency f_m lower than that of the checkerboard pattern f_c. The sampling of non-integer number of cycles of these square planar waves causes spectral leakage and affects the estimation accuracy of the checkerboard's base frequency (coordinates of point A and point B in Fig. 7). Specifically, leakage comes from the peak at a multiple of f_m that is close to f_c. In other words, embedding marker fields into checkerboards will bring errors to non-integer period coordinates of point A and point B in Fig. 7 through the high-order harmonics. The angle θ is derived indirectly through the two vectors given by Eq. (6) which propagates and magnifies errors in coordinates of point A and point B in Fig. 7, hence the impact on θ accuracy appears more substantial than in x and y axes.

One thing to note is that the errors are not the same for translations in x and y axes for all 3 coding schemes, especially HMF. This difference is more evident for smaller image sizes. The asymmetry in x and y axes

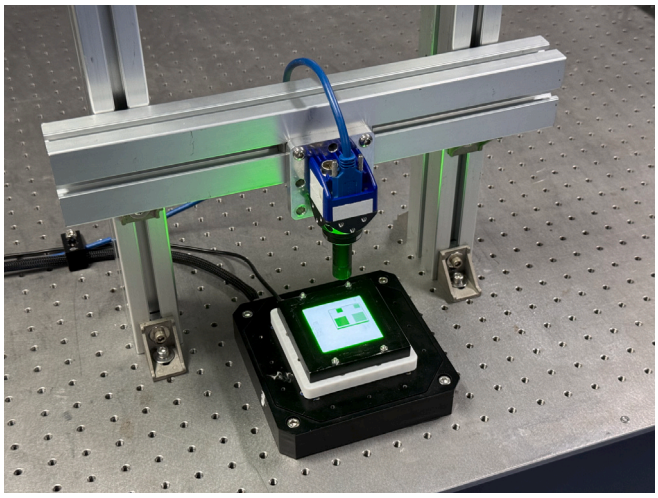


Fig. 16. Experimental setup for absolute 3-DoF translation and rotation measurements using the piezo nanopositioning stage.

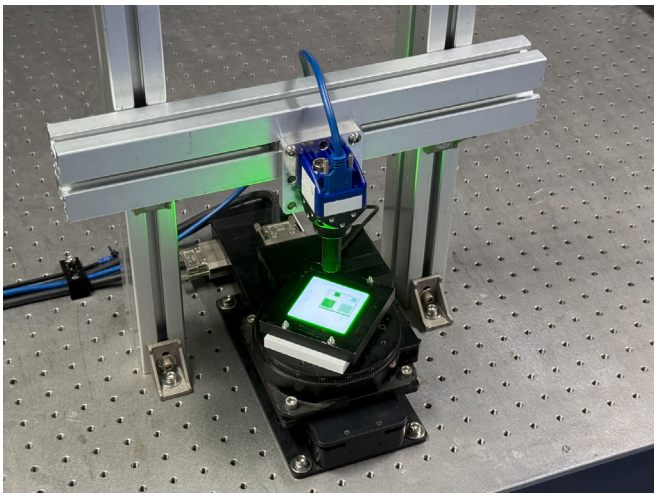


Fig. 17. Experimental setup for absolute 3-DoF translation and rotation measurements using stepper-motor driven stages.

is due to the asymmetric nature of the coding schemes. Despite that the 1 s and 0 s tend to be distributed evenly within all 3 coding schemes mentioned, in a finite region, asymmetry in x and y axes exists and intensifies as the observation window narrows, explaining asymmetry in x and y especially as acquisition size becomes rather small. Also, since HMF is generated by 1D sequence while DBT and UMF by 2D, the asymmetry is more prevalent in HMF than in DBT and UMF.

To isolate the impact of the embedded codes, we analyzed the relationship between the measured local code density and pose estimation accuracy. The results, shown in Fig. 15, indicate that for the narrow density ranges produced by these specific configurations, the influence of small density variations is minimal and is outweighed by other factors. The data does, however, highlight key structural differences between the markers. As expected, DBT occupies a higher code density range due to the overhead of its explicit orientation codes.

Most significantly, a clear positive trend between local code density and RMSE values can be observed, suggesting that HMF and UMF gains advantage by lesser embedded blocks, confirming our observation in Fig. 14. However, some subtle differences exist between the compared marker fields, which could suggest that the structure of the marker field is still an accuracy impact factor.

5. Experiments

In this section, experiments with an absolute 3-DoF planar pose measurement setup are described. A checkerboard pattern with an embedded HMF of window array size 17+17 is etched onto a small glass plate. For such a window array size, HMF can have a field size of over $10{,}000 \times 10{,}000$ bits. However, only the top-left 100×100 bits of this field is used. The checkerboard pattern has a block size of 100×100 μm and a total size of 10×10 mm. If the entire marker field is used, the checkerboard pattern can be as large as 1×1 m, making large-area absolute translation and rotation measurements possible. The glass checkerboard is illuminated with an LED backlight for high image contrast. Image acquisition is done with a black-and-white industrial camera (Matrix Vision mvBlueFOX3-2204G) with 4512×4512 pixels. The experiments are conducted by using a 6-axis nanopositioning stage (Physik Instrumente P−562.6CD) and a set of stepper-motor driven linear and rotation stages (OptoSigma OSMS26-50X & OptoSigma OSMS-120YAW). The key specifications of these stages are listed in Table 3, and the experiment setups are shown in Figs. 16 and 17.

In the first experiment, the glass checkerboard was mounted on the nanopositioning stage to evaluate the performance limits of the proposed method. The stage was first programmed to translate over a range of 1 μm in steps of 20 nm. Subsequently, it was programmed to rotate 100 μrad in increments of 2 μrad. The measured results of these translations and rotations are presented in Fig. 18. Analysis of the measurement data yields a Root Mean Square Error (RMSE) of 8.5 nm for translation and 1.7 μrad for rotation over the tested ranges. Since the accuracy of the nanopositioning stage is ±2 nm for translation and ±0.1 μrad for rotation, the above RMSEs are primarily the errors of the proposed method itself and are the performance limit in terms of accuracy.

Table 3
Specifications of motion stages.

Model name	Axis	Resolution	Accuracy	Travel
P-562.6CD	x	1 nm	±2 nm	200 μm
	y	1 nm	±2 nm	200 μm
	z	1 nm	±3 nm	200 μm
	R_x	0.1 μrad	±0.1 μrad	0.5 mrad
	R_y	0.1 μrad	±0.1 μrad	0.5 mrad
	R_z	0.1 μrad	±0.15 μrad	0.5 mrad
OSMS26-50X	x	4 μm	5 μm	50 mm
OSMS-120YAW	R_z	0.005°	0.1°	360°

In the second experiment, the glass checkerboard is mounted on the rotation stage, which is in turn mounted on the linear stage. The stages are programmed to translate 1000 μm in steps of 20 μm and rotate 180° in steps of 4°. Fig. 19 shows the translation and rotation errors of the stages, which are slightly larger than ±4 μm in translation and approximately 1 mrad or 0.05° in rotation over the entire ranges of motion. These errors are reasonable compared to the specifications of the stages shown in Table 3. Crucially, these results demonstrate that the proposed marker field successfully achieves orientation decoding, as accurate measurements were consistently obtained even for rotations surpassing 90°, confirming its effectiveness across the full angular range.

6. Conclusion

We have introduced a new type of orientable fiducial marker field called Homomorphism Marker Field (HMF). We detailed its construction procedure, which remarkably enables the efficient generation and decoding of exceptionally large marker fields. A defining characteristic of HMF is that it leverages Lempel's D-homomorphism and aperiodic orientable sequences (AOS), utilizing a local cross-shaped window to

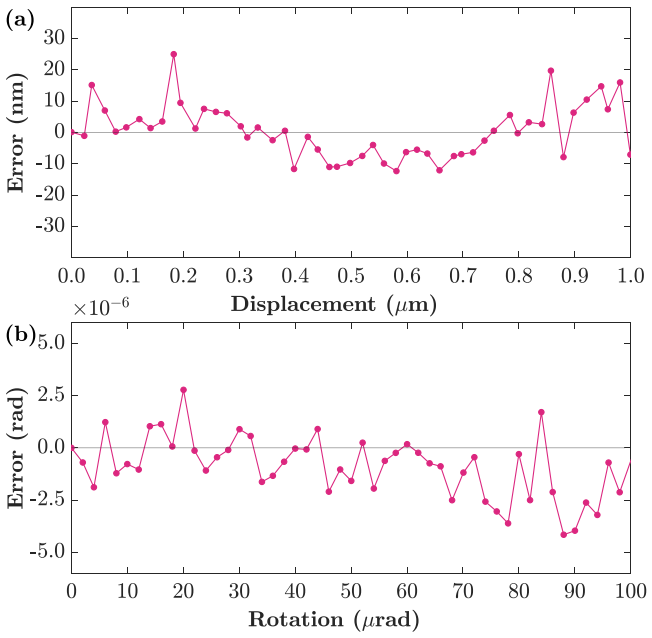


Fig. 18. Errors of translation and rotation measurement. (a) Error of translation measurement. (b) Error of rotation measurement.

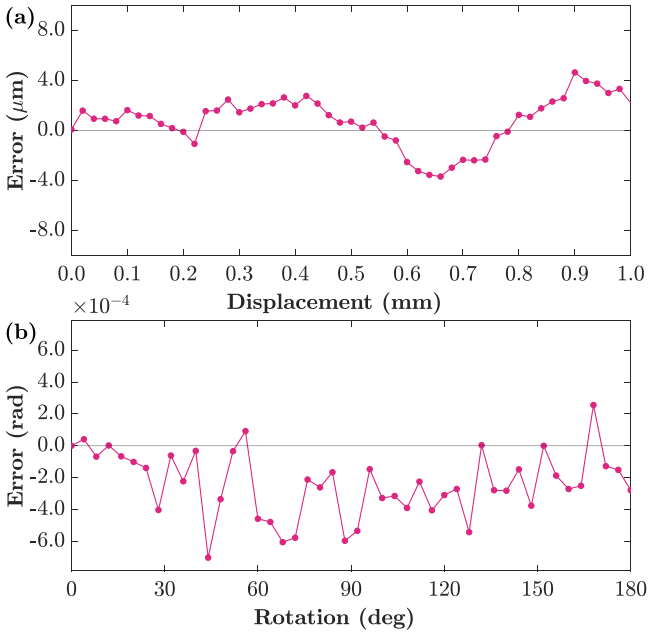


Fig. 19. Measured translation and rotation errors of the linear and rotation stages. (a) Translation error. (b) Rotation error.

uniquely determine its position and orientation within the field. Simulation results further highlight HMF's performance, demonstrating significantly faster decoding speeds and near-linear decoding time growth with marker field size compared to UMF and DBT under comparable conditions.

As an application, HMF is embedded in a checkerboard pattern and used for absolute 3-DoF translation and rotation measurement, which is achieved by combining frequency and phase information extracted via a 2D-IpDFT method with the decoding result of HMF. Experimental results show errors of 8.5 nm for translation and 1.7 μrad for rotation measurement respectively, with the HMF's effective orientation decoding supporting measurements across the full angular range. Crucially, since the HMF configuration used in these experiments supports marker fields up to 1 × 1 m in size, these results demonstrate its potential for enabling absolute positioning with nanometer-level accuracy across meter-scale planar areas.

The proposed HMF has many other potential applications in camera localization/calibration, robot/vehicle navigation, etc. The AOS in the construction of HMF can be replaced by other sequences of different properties. If orientation is fixed, replacing AOS with a full-length de Bruijn sequence in each dimension will vastly expand the HMF coverage size. Future work includes characterizing the exact properties of homomorphism and product construction mapping in k-alphabet HMF construction and optimizing the cross-shaped window into more compact geometries.

CRediT authorship contribution statement

Zian Mao: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Jiayue Ding:** Writing – original draft, Visualization, Methodology, Formal analysis, Data curation. **Guowei Shi:** Methodology, Investigation, Data curation. **Peisen Huang:** Writing – review & editing, Supervision, Resources, Project administration, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors thank UM-SJTU Joint Institute for its support of this research.

Data availability

Data will be made available on request.

References

[1] C.-P. Tung, A.C. Kak, Integrating sensing, task planning, and execution for robotic assembly, IEEE Trans. Robot. Autom. 12 (2) (1996) 187–201.
[2] P. Sun, F. Zhou, J. Sun, L. Wang, High-accuracy three-dimensional measurement based on multi-directional cooperative target with weighted SfM algorithm, Measurement 172 (2021) 108955.
[3] F. Yuan, Z. Xia, B. Tang, Z. Yin, X. Shao, X. He, Calibration accuracy evaluation method for multi-camera measurement systems, Measurement 242 (2025) 116311, http://dx.doi.org/10.1016/j.measurement.2024.116311, URL https://www.sciencedirect.com/science/article/pii/S0263224124021961.
[4] J. Hong, S.-H. Sim, J. Lee, H. Kim, S. Cho, J. Lee, Long-term displacement measurement system for bridge bearing capable of camera reposition, Measurement 235 (2024) 114921, http://dx.doi.org/10.1016/j.measurement.2024.114921, URL https://www.sciencedirect.com/science/article/pii/S0263224124008066.
[5] J. Burns, C. Mitchell, Coding schemes for two-dimensional position sensing, 1993.
[6] A. Herout, I. Szentandrasi, M. Zacharias, M. Dubska, R. Kajan, Five shades of grey for fast and reliable camera pose estimation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2013.
[7] I. Szentandrási, M. Zachariáš, J. Havel, A. Herout, M. Dubská, R. Kajan, Uniform marker fields: Camera localization by orientable De Bruijn tori, in: 2012 IEEE International Symposium on Mixed and Augmented Reality, ISMAR, 2012, pp. 319–320, http://dx.doi.org/10.1109/ISMAR.2012.6402593.
[8] Z. Gao, M. Zhu, J. Yu, A novel camera calibration pattern robust to incomplete pattern projection, IEEE Sens. J. 21 (8) (2021) 10051–10060.
[9] N. Hagbi, O. Bergig, J. El-Sana, M. Billinghurst, Shape recognition and pose estimation for mobile augmented reality, in: 2009 8th IEEE International Symposium on Mixed and Augmented Reality, 2009, pp. 65–71, http://dx.doi.org/10.1109/ISMAR.2009.5336498.
[10] D. Schüsselbauer, A. Schmid, R. Wimmer, Dothraki: Tracking tangibles atop tabletops through De-Bruijn Tori, in: Proceedings of the Fifteenth International Conference on Tangible, Embedded, and Embodied Interaction, 2021, pp. 1–10.

[11] R. Adámek, M. Brablc, P. Vávra, B. Dobossy, M. Formánek, F. Radil, Analytical models for pose estimate variance of planar fiducial markers for mobile robot localisation, Sensors 23 (12) (2023) 5746.
[12] F. Li, X. Shang, Q. Tao, T. Zhang, G. Shi, Y. Niu, Single-shot depth sensing with pseudo two-dimensional sequence coded discrete binary pattern, IEEE Sens. J. 21 (9) (2021) 11075–11083.
[13] N.G. De Bruijn, A combinatorial problem, Proc. Sect. Sci. Koninklijke Nederlandse Akademie Wetenschappen Amsterdam 49 (7) (1946) 758–764.
[14] M. Zhu, B. He, J. Yu, F. Yuan, J. Liu, HydraMarker: Efficient, flexible, and multifold marker field generation, IEEE Trans. Pattern Anal. Mach. Intell. 45 (5) (2023) 5849–5861, http://dx.doi.org/10.1109/TPAMI.2022.3212862.
[15] A.M. Bruckstein, T. Etzion, R. Giryes, N. Gordon, R.J. Holt, D. Shuldiner, Simple and robust binary self-location patterns, IEEE Trans. Inform. Theory 58 (7) (2012) 4884–4889, http://dx.doi.org/10.1109/TIT.2012.2191699.
[16] H. Yu, Q. Wan, Z. Mu, Y. Du, L. Liang, Novel nano-scale absolute linear displacement measurement based on grating projection imaging, Measurement 182 (2021) 109738.
[17] B. Gordon, On the existence of perfect maps (Corresp.), IEEE Trans. Inform. Theory 12 (4) (1966) 486–487.
[18] S.W. Golomb, Shift Register Sequences, Aegean Park Press, USA, 1981.
[19] F. MacWilliams, N. Sloane, Pseudo-random sequences and arrays, Proc. IEEE 64 (12) (1976) 1715–1729, http://dx.doi.org/10.1109/PROC.1976.10411.
[20] K. Martin, Z. Dai, M. Robshaw, P. Wild, Orientable sequences, in: Proceedings of the 3rd IMA Conference on Cryptography and Coding, 1993, pp. 97–115.
[21] C.J. Mitchell, P.R. Wild, Constructing orientable sequences, IEEE Trans. Inform. Theory 68 (7) (2022) 4782–4789.
[22] A. Alhakim, C.J. Mitchell, J. Szmidt, P.R. Wild, Orientable sequences over non-binary alphabets, Cryptogr. Commun. (2024) 1–18.
[23] A. Lempel, On a homomorphism of the de Bruijn Graph and its applications to the design of feedback shift registers, IEEE Trans. Comput. C-19 (12) (1970) 1204–1209, http://dx.doi.org/10.1109/T-C.1970.222859.
[24] A.V. Oppenheim, J.S. Lim, The importance of phase in signals, Proc. IEEE 69 (5) (1981) 529–541.
[25] Z.-H. Chen, P. Huang, A vision-based method for planar position measurement, Meas. Sci. Technol. 27 (12) (2016) 125018.
[26] Q. Cheng, P. Huang, Camera calibration based on phase estimation, IEEE Trans. Instrum. Meas. 72 (2022) 1–9.
[27] Q. Cheng, T. Xu, P. Huang, Systematic error analysis and parameter design of a vision-based phase estimation method for ultra-precision positioning, Opt. Express 30 (5) (2022) 8330–8344.
[28] G. Shi, P. Huang, Subsignals technique for estimating parameters of signals, Mech. Syst. Signal Process. 224 (2025) 112060, http://dx.doi.org/10.1016/j.ymssp.2024.112060, URL https://www.sciencedirect.com/science/article/pii/S0888327024009580.
[29] G. Shi, Z. Mao, J. Ding, P. Huang, Subsignal technique for estimating parameters of signals using symmetric windows, Opt. Express 33 (11) (2025) 23400–23414, http://dx.doi.org/10.1364/OE.561813, URL https://opg.optica.org/oe/abstract.cfm?URI=oe-33-11-23400.
[30] G. Shi, Z. Mao, J. Ding, P. Huang, 2-D interpolated discrete Fourier transform for High-Accuracy In-Plane displacement and rotation measurement, 2024, pp. 1–6, http://dx.doi.org/10.1109/IST63414.2024.10759177.
[31] J. Borkowski, D. Kania, J. Mroczka, Interpolated-DFT-based fast and accurate frequency estimation for the control of power, IEEE Trans. Ind. Electron. 61 (12) (2014) 7026–7034.